Intermediate (watch the model choose between two tools) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangChain/Python). Read Basic first even if you already know this — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

You'll build two separate tools that sound almost the same — like one that gets today's weather, and one that gets a multi-day forecast.

Register both at once, and ask something deliberately vague, like "what's the weather like" — nothing that clearly says "today" or "this week."

Run that same question several times, and write down, each time, which tool got picked. You're not trying to "fix" anything yet — just observe.

Things to use:

- Two `@tool`-decorated functions with similar one-line docstrings.
- `model_with_tools = ChatOpenAI().bind_tools([get_weather, get_forecast])`
- A loop that calls `.invoke("what's the weather like")` 5 times.
- Each time, print `response.tool_calls[0]["name"]` if there is a call, or `"none"` if there isn't.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

`bind_tools([tool_a, tool_b])` hands the model both descriptions at once, and tool selection happens entirely inside the model — your code has no `if` / `else` deciding which one to call. When both descriptions could plausibly answer the same question, the choice becomes sensitive to small wording differences in the prompt, and can vary run to run even at `temperature=0` for tool selection specifically (temperature affects text generation more predictably than tool-choice, which is worth confirming for yourself here).

The point of running the same ambiguous prompt 5 times isn't to find "the bug" — it's to see, directly, that an overlapping pair of descriptions produces a real, measurable split, not a single wrong answer you can just patch.

Write the two docstrings deliberately similar — e.g. `"Get the current weather for a city."` and `"Get the weather forecast for a city."` — close enough that a human skimming them might also be unsure. Tally the picks with a plain dictionary, like `{"get_weather": 0, "get_forecast": 0, "none": 0}`, and print a real number at the end, not just "it seemed to favor one."

A 5-run split like `{"get_weather": 3, "get_forecast": 2}` raises a harder question than "which one is right": is that split itself a problem, or is it just what a genuinely unclear prompt should produce? The prompt genuinely doesn't say whether "today" or "this week" was meant — a perfect model might legitimately split its answers too. The fix isn't always "make the split go away" — sometimes it's "recognize the prompt actually needs a follow-up question."

Difference between Basic and Intermediate: Basic gets you a real, measured split from one trial with a plain dictionary. Intermediate wraps the trial in a reusable, per-run-logging function that returns the counts dictionary (or a `collections.Counter`, which starts any missing key at 0 — the Solution's Approach 2) — this is the depth the exercise's Solution is written at.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define get_weather - "get the current weather for a city"
define get_forecast - "get the weather forecast for a city"

register both on a model

counts = {get_weather: 0, get_forecast: 0, none: 0}

repeat 5 times:
    ask "what's the weather like"
    look at which tool (if any) was picked
    add one to that count

print counts
```

Here's almost the whole thing — just try running it:

```
# tool_selection_practice.py — Intermediate section
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny in {city}"

@tool
def get_forecast(city: str) -> str:
    """Get the weather forecast for a city."""
    return f"Rainy later this week in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather, get_forecast])

counts = {"get_weather": 0, "get_forecast": 0, "none": 0}
for i in range(5):
    response = model_with_tools.invoke("what's the weather like")
    if response.tool_calls:
```

```
        picked = response.tool_calls[0]["name"]
    else:
        picked = "none"
    counts[picked] = counts.get(picked, 0) + 1

print(counts)
```

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
@tool
def get_weather(city: str) -> str: "Get the current weather for a city."

@tool
def get_forecast(city: str) -> str: "Get the weather forecast for a city."

function get_model_with_tools():
    return ChatOpenAI(...).bind_tools([get_weather, get_forecast])

function run_selection_trial(prompt, runs=5) -> dict:
    model_with_tools = get_model_with_tools()
    counts = {"get_weather": 0, "get_forecast": 0, "none": 0}
    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            picked = response.tool_calls[0]["name"]
        else:
            picked = "none"
        print(f"Run {run_number}: picked {picked}")
        counts[picked] = counts[picked] + 1
    return counts
```

Wire this into a full script with `if __name__ == "__main__":`, then compare against the Solution.

Difference between Basic and Intermediate: same underlying trial (register, ask 5 times, count) at 2 completeness levels — Basic's version prints one dictionary and stops, Intermediate wraps it in a reusable, per-run-logging function.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution